

NumPy Complete Reference Manual

Every topic and subtopic explained in two lines, with four runnable examples and real outputs each.

By PRAVEEN VIJAYAPURAM

1. Introduction to NumPy

1.1 Importing NumPy

NumPy is imported using the standard alias `np`. This alias is used throughout the entire NumPy ecosystem and documentation.

Question 1: Write a program to import NumPy and print the installed version.

Input:

```
import numpy as np
print("NumPy successfully loaded version:", np.__version__)
```

Output:

```
NumPy successfully loaded version: 2.4.4
```

Question 2: Write a program to check whether NumPy has been imported successfully.

Input:

```
import numpy as np
print("Is NumPy imported?", 'np' in dir())
```

Output:

```
Is NumPy imported? True
```

Question 3: Write a program to print the name of the NumPy module.

Input:

```
import numpy as np
print("NumPy module name:", np.__name__)
```

Output:

```
NumPy module name: numpy
```

Question 4: Write a program to print the file path of the installed NumPy module.

Input:

```
import numpy as np
print("NumPy file location:", np.__file__[-30:])
```

Output:

```
NumPy file location: ist-packages/numpy/__init__.py
```

1.2 Creating an Array

A Python list can be converted into a NumPy ndarray using `np.array()`. The ndarray is faster and more memory-efficient than a plain list.

Question 1: Write a program to create a NumPy array from a list of integers.

Input:

```
import numpy as np
my_list = [5, 10, 15, 20]
my_array = np.array(my_list)
print("Created Array:", my_array)
```

Output:

```
Created Array: [ 5 10 15 20]
```

Question 2: Write a program to create a NumPy array from a list of strings.

Input:

```
import numpy as np
names_list = ["a", "b", "c"]
names_array = np.array(names_list)
print("String Array:", names_array)
```

Output:

```
String Array: ['a' 'b' 'c']
```

Question 3: Write a program to create a 2D NumPy array from a nested list.

Input:

```
import numpy as np
nested_list = [[1, 2], [3, 4]]
matrix_array = np.array(nested_list)
```

```
print("2D Array:\n", matrix_array)
```

Output:

```
2D Array:  
[[1 2]  
 [3 4]]
```

Question 4: Write a program to create a NumPy array from a list of floating-point numbers.

Input:

```
import numpy as np  
float_list = [1.1, 2.2, 3.3]  
float_array = np.array(float_list)  
print("Float Array:", float_array)
```

Output:

```
Float Array: [1.1 2.2 3.3]
```

2. Array Creation Functions

2.1 np.zeros()

`np.zeros()` builds a new array of a given shape filled entirely with the value 0. It is often used to pre-allocate space before filling in real data.

Question 1: Write a program to create a 3x4 array filled with zeros.

Input:

```
import numpy as np  
zeros_array = np.zeros((3, 4))  
print(zeros_array)
```

Output:

```
[[0. 0. 0. 0.]  
 [0. 0. 0. 0.]  
 [0. 0. 0. 0.]]
```

Question 2: Write a program to create a 1D array of five zeros.

Input:

```
import numpy as np
zeros_1d = np.zeros(5)
print(zeros_1d)
```

Output:

```
[0. 0. 0. 0. 0.]
```

Question 3: Write a program to create a 2x2 integer array of zeros.

Input:

```
import numpy as np
zeros_int = np.zeros((2, 2), dtype=int)
print(zeros_int)
```

Output:

```
[[0 0]
 [0 0]]
```

Question 4: Write a program to create a 3D array of zeros and print its shape.

Input:

```
import numpy as np
zeros_3d = np.zeros((2, 2, 2))
print(zeros_3d.shape)
```

Output:

```
(2, 2, 2)
```

2.2 np.ones()

`np.ones()` builds a new array of a given shape filled entirely with the value 1. It is commonly used as a starting point for masks or weight matrices.

Question 1: Write a program to create a 2x3 array filled with ones.

Input:

```
import numpy as np
ones_array = np.ones((2, 3))
print(ones_array)
```

Output:

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

Question 2: Write a program to create a 1D array of four ones.

Input:

```
import numpy as np
ones_1d = np.ones(4)
print(ones_1d)
```

Output:

```
[1. 1. 1. 1.]
```

Question 3: Write a program to create a 3x3 integer array of ones.

Input:

```
import numpy as np
ones_int = np.ones((3, 3), dtype=int)
print(ones_int)
```

Output:

```
[[1 1 1]
 [1 1 1]
 [1 1 1]]
```

Question 4: Write a program to create an array of ones and scale every value by 5.

Input:

```
import numpy as np
ones_scaled = np.ones((2, 2)) * 5
print(ones_scaled)
```

Output:

```
[[5. 5.]  
 [5. 5.]]
```

2.3 np.full()

np.full() creates an array of a chosen shape where every single element is set to one specific constant value. It saves writing a loop to fill an array manually.

Question 1: Write a program to create a 2x2 array filled with the value 99.

Input:

```
import numpy as np  
full_array = np.full((2, 2), 99)  
print(full_array)
```

Output:

```
[[99 99]  
 [99 99]]
```

Question 2: Write a program to create a 1D array of five sevens.

Input:

```
import numpy as np  
full_row = np.full(5, 7)  
print(full_row)
```

Output:

```
[7 7 7 7 7]
```

Question 3: Write a program to create a 2x3 array filled with a float constant.

Input:

```
import numpy as np  
full_float = np.full((2, 3), 3.14)  
print(full_float)
```

Output:

```
[[3.14 3.14 3.14]  
 [3.14 3.14 3.14]]
```

Question 4: Write a program to create a 1D array filled with a text value.

Input:

```
import numpy as np
full_text = np.full(3, "hi")
print(full_text)
```

Output:

```
['hi' 'hi' 'hi']
```

2.4 np.arange()

np.arange() generates a sequence of numbers starting at a value, stopping before a limit, using a fixed step size. It behaves like Python's range() but returns an array.

Question 1: Write a program to generate a range of numbers from 10 to 35 with a step of 5.

Input:

```
import numpy as np
range_array = np.arange(10, 35, 5)
print("Generated Range Array:", range_array)
```

Output:

```
Generated Range Array: [10 15 20 25 30]
```

Question 2: Write a program to generate a simple range of numbers from 0 to 4.

Input:

```
import numpy as np
simple_range = np.arange(5)
print(simple_range)
```

Output:

```
[0 1 2 3 4]
```

Question 3: Write a program to generate a countdown sequence using a negative step.

Input:

```
import numpy as np
countdown = np.arange(10, 0, -2)
print(countdown)
```

Output:

```
[10  8  6  4  2]
```

Question 4: Write a program to generate a range of numbers using a decimal step size.

Input:

```
import numpy as np
decimal_step = np.arange(0, 1, 0.25)
print(decimal_step)
```

Output:

```
[0.  0.25  0.5  0.75]
```

2.5 np.linspace()

np.linspace() returns a chosen number of evenly spaced values between a start and stop point, both inclusive. This is useful for plotting smooth graphs.

Question 1: Write a program to generate four evenly spaced values between 0 and 9.

Input:

```
import numpy as np
linspace_array = np.linspace(0, 9, 4)
print("Linear Spaced Array:", linspace_array)
```

Output:

```
Linear Spaced Array: [0.  3.  6.  9.]
```

Question 2: Write a program to generate five evenly spaced values between 0 and 1.

Input:

```
import numpy as np
five_points = np.linspace(0, 1, 5)
print(five_points)
```

Output:

```
[0.  0.25  0.5  0.75  1. ]
```

Question 3: Write a program to generate three evenly spaced values between -10 and 10.

Input:

```
import numpy as np
neg_to_pos = np.linspace(-10, 10, 3)
print(neg_to_pos)
```

Output:

```
[-10.  0.  10.]
```

Question 4: Write a program to generate evenly spaced values while excluding the endpoint.

Input:

```
import numpy as np
no_endpoint = np.linspace(0, 10, 5, endpoint=False)
print(no_endpoint)
```

Output:

```
[0.  2.  4.  6.  8.]
```

2.6 np.eye() (Identity Matrix)

np.eye() builds a square matrix with 1s on the main diagonal and 0s everywhere else. Identity matrices are the neutral element for matrix multiplication.

Question 1: Write a program to create a 3x3 identity matrix.

Input:

```
import numpy as np
identity_matrix = np.eye(3)
```

```
print(identity_matrix)
```

Output:

```
[[1.  0.  0.]  
 [0.  1.  0.]  
 [0.  0.  1.]]
```

Question 2: Write a program to create a 2x2 identity matrix.

Input:

```
import numpy as np  
small_identity = np.eye(2)  
print(small_identity)
```

Output:

```
[[1.  0.]  
 [0.  1.]]
```

Question 3: Write a program to create a 5x5 integer identity matrix.

Input:

```
import numpy as np  
big_identity = np.eye(5, dtype=int)  
print(big_identity)
```

Output:

```
[[1 0 0 0 0]  
 [0 1 0 0 0]  
 [0 0 1 0 0]  
 [0 0 0 1 0]  
 [0 0 0 0 1]]
```

Question 4: Write a program to create a rectangular identity-style matrix.

Input:

```
import numpy as np  
rect_identity = np.eye(3, 4)  
print(rect_identity)
```

Output:

```
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]]
```

3. Array Attributes

3.1 shape

The `.shape` attribute reports the size of an array along each of its axes as a tuple. It tells you how many rows and columns (and deeper dimensions) the array has.

Question 1: Write a program to print the shape of a 3x2 matrix.

Input:

```
import numpy as np
matrix = np.array([[10, 20], [30, 40], [50, 60]])
print("Array Shape Layout:", matrix.shape)
```

Output:

```
Array Shape Layout: (3, 2)
```

Question 2: Write a program to print the shape of a 1D vector.

Input:

```
import numpy as np
vector = np.array([1, 2, 3, 4])
print(vector.shape)
```

Output:

```
(4,)
```

Question 3: Write a program to print the shape of a 3D array.

Input:

```
import numpy as np
cube = np.zeros((2, 3, 4))
print(cube.shape)
```

Output:

```
(2, 3, 4)
```

Question 4: Write a program to print the shape of a single-row 2D array.

Input:

```
import numpy as np
single_row = np.array([[1, 2, 3]])
print(single_row.shape)
```

Output:

```
(1, 3)
```

3.2 ndim

The `.ndim` attribute returns how many dimensions (axes) an array has. A flat list is 1 dimension, a table/matrix is 2 dimensions, and so on.

Question 1: Write a program to print the number of dimensions of a 1D array.

Input:

```
import numpy as np
arr_1d = np.array([1, 2, 3])
print("1D Array Dimensions:", arr_1d.ndim)
```

Output:

```
1D Array Dimensions: 1
```

Question 2: Write a program to print the number of dimensions of a 2D array.

Input:

```
import numpy as np
arr_2d = np.array([[1, 2], [3, 4]])
print(arr_2d.ndim)
```

Output:

```
2
```

Question 3: Write a program to print the number of dimensions of a 3D array.

Input:

```
import numpy as np
arr_3d = np.zeros((2, 2, 2))
print(arr_3d.ndim)
```

Output:

3

Question 4: Write a program to print the number of dimensions of a scalar-like array.

Input:

```
import numpy as np
scalar_like = np.array(5)
print(scalar_like.ndim)
```

Output:

0

3.3 size

The `.size` attribute returns the total count of individual elements stored in the array, regardless of how many dimensions it has. It is equal to the product of all the shape values.

Question 1: Write a program to print the total number of elements in a 4x5 array.

Input:

```
import numpy as np
matrix = np.zeros((4, 5))
print("Total Elements:", matrix.size)
```

Output:

Total Elements: 20

Question 2: Write a program to print the total number of elements in a 1D array.

Input:

```
import numpy as np
vector = np.array([1, 2, 3])
print(vector.size)
```

Output:

3

Question 3: Write a program to print the total number of elements in a 3D array.

Input:

```
import numpy as np
cube = np.ones((2, 3, 2))
print(cube.size)
```

Output:

12

Question 4: Write a program to print the size of an empty array.

Input:

```
import numpy as np
empty_arr = np.array([])
print(empty_arr.size)
```

Output:

0

3.4 dtype

The `.dtype` attribute reveals the underlying data type used to store each element, such as `int64` or `float64`. All elements in a NumPy array must share the same dtype.

Question 1: Write a program to print the data type of a float array.

Input:

```
import numpy as np
float_arr = np.array([1.5, 2.7, 3.1])
print("Element Data Type:", float_arr.dtype)
```

Output:

Element Data Type: float64

Question 2: Write a program to print the data type of an integer array.

Input:

```
import numpy as np
int_arr = np.array([1, 2, 3])
print(int_arr.dtype)
```

Output:

int64

Question 3: Write a program to print the data type of a boolean array.

Input:

```
import numpy as np
bool_arr = np.array([True, False])
print(bool_arr.dtype)
```

Output:

bool

Question 4: Write a program to force an array to a specific float data type.

Input:

```
import numpy as np
forced_arr = np.array([1, 2, 3], dtype=np.float32)
print(forced_arr.dtype)
```

Output:

float32

4. Indexing and Slicing

4.1 1D Indexing

Elements in a one-dimensional array are accessed with a single index in square brackets, starting at 0. Negative indices count backward from the end of the array.

Question 1: Write a program to access the first element of a 1D array.

Input:

```
import numpy as np
arr = np.array([100, 200, 300, 400])
print("First Element:", arr[0])
```

Output:

```
First Element: 100
```

Question 2: Write a program to access the last element of a 1D array using a negative index.

Input:

```
import numpy as np
arr = np.array([100, 200, 300, 400])
print("Last Element:", arr[-1])
```

Output:

```
Last Element: 400
```

Question 3: Write a program to access the third element of a 1D array.

Input:

```
import numpy as np
arr = np.array([5, 10, 15, 20, 25])
print("Third Element:", arr[2])
```

Output:

```
Third Element: 15
```

Question 4: Write a program to update a single element of a 1D array.

Input:

```
import numpy as np
arr = np.array([1, 2, 3])
arr[1] = 99
print("After Update:", arr)
```

Output:

```
After Update: [ 1 99 3]
```

4.2 2D Indexing

Elements in a two-dimensional array are accessed with two indices, `matrix[row, column]`. Each index still starts counting from 0.

Question 1: Write a program to access a specific element of a 2D array using row and column indices.

Input:

```
import numpy as np
matrix_2d = np.array([[7, 8, 9], [4, 5, 6]])
print("Element at [1, 2]:", matrix_2d[1, 2])
```

Output:

```
Element at [1, 2]: 6
```

Question 2: Write a program to access the top-left element of a 2D array.

Input:

```
import numpy as np
matrix_2d = np.array([[1, 2], [3, 4]])
print("Top-left Element:", matrix_2d[0, 0])
```

Output:

```
Top-left Element: 1
```

Question 3: Write a program to access an entire row of a 2D array.

Input:

```
import numpy as np
matrix_2d = np.array([[1, 2, 3], [4, 5, 6]])
print("Whole Row 0:", matrix_2d[0])
```

Output:

```
Whole Row 0: [1 2 3]
```

Question 4: Write a program to update a single element of a 2D array.

Input:

```
import numpy as np
matrix_2d = np.array([[1, 2], [3, 4]])
matrix_2d[1, 0] = 100
print("After Update:\n", matrix_2d)
```

Output:

```
After Update:
[[ 1  2]
 [100 4]]
```

4.3 Slicing 2D Arrays

Slicing with the colon operator extracts a whole row, column, or rectangular block from a matrix at once. A bare colon means take everything along that axis.

Question 1: Write a program to extract the first column of a matrix using slicing.

Input:

```
import numpy as np
grid = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("First Column Vector:", grid[:, 0])
```

Output:

```
First Column Vector: [1 4 7]
```

Question 2: Write a program to extract the second row of a matrix using slicing.

Input:

```
import numpy as np
grid = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Second Row:", grid[1, :])
```

Output:

```
Second Row: [4 5 6]
```

Question 3: Write a program to extract the top-left 2x2 block of a matrix.

Input:

```
import numpy as np
grid = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Top-left 2x2 Block:\n", grid[:2, :2])
```

Output:

```
Top-left 2x2 Block:
[[1 2]
 [4 5]]
```

Question 4: Write a program to extract the last two rows of a matrix.

Input:

```
import numpy as np
grid = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Last Two Rows:\n", grid[1:, :])
```

Output:

```
Last Two Rows:
[[4 5 6]
 [7 8 9]]
```

4.4 Boolean Indexing (Filtering)

Boolean indexing uses a True/False condition to pick out only the array elements that satisfy it, with no explicit loop needed. The condition itself creates a mask array of the same shape.

Question 1: Write a program to filter array elements greater than 10.

Input:

```
import numpy as np
data = np.array([5, 12, 7, 18, 3])
filtered = data[data > 10]
print("Elements greater than 10:", filtered)
```

Output:

```
Elements greater than 10: [12 18]
```

Question 2: Write a program to filter out the even numbers from an array.

Input:

```
import numpy as np
data = np.array([1, 2, 3, 4, 5, 6])
evens = data[data % 2 == 0]
print("Even Numbers:", evens)
```

Output:

```
Even Numbers: [2 4 6]
```

Question 3: Write a program to filter out array elements not equal to a given value.

Input:

```
import numpy as np
data = np.array([10, 20, 30, 40])
mask = data != 20
print("Not Equal to 20:", data[mask])
```

Output:

```
Not Equal to 20: [10 30 40]
```

Question 4: Write a program to filter out the positive numbers from an array.

Input:

```
import numpy as np
data = np.array([-5, 3, -2, 8, -1])
positives = data[data > 0]
print("Positive Numbers:", positives)
```

Output:

```
Positive Numbers: [3 8]
```

5. Data Types & Conversion

5.1 astype()

The `astype()` method returns a new array with every element converted to a specified data type. The original array is left unchanged since a copy is created.

Question 1: Write a program to convert a float array to 32-bit integers.

Input:

```
import numpy as np
floats = np.array([10.9, 20.1, 30.5])
integers = floats.astype(np.int32)
print("Converted Elements:", integers)
```

Output:

```
Converted Elements: [10 20 30]
```

Question 2: Write a program to convert an integer array to floats.

Input:

```
import numpy as np
ints = np.array([1, 2, 3])
floats = ints.astype(float)
print(floats)
```

Output:

```
[1. 2. 3.]
```

Question 3: Write a program to convert a numeric array to booleans.

Input:

```
import numpy as np
nums = np.array([0, 1, 2])
bools = nums.astype(bool)
print(bools)
```

Output:

```
[False True True]
```

Question 4: Write a program to convert a numeric array to strings.

Input:

```
import numpy as np
nums = np.array([1, 2, 3])
text = nums.astype(str)
```

```
print(text)
```

Output:

```
['1' '2' '3']
```

6. Basic Arithmetic

6.1 Element-wise Operations

Arithmetic operators like +, -, *, and / apply automatically to every matching pair of elements across two same-shaped arrays. No manual looping is required for these calculations.

Question 1: Write a program to add two arrays element-wise.

Input:

```
import numpy as np
arr_a = np.array([1, 2, 3])
arr_b = np.array([10, 20, 30])
print("Element-wise Addition:", arr_a + arr_b)
```

Output:

```
Element-wise Addition: [11 22 33]
```

Question 2: Write a program to subtract two arrays element-wise.

Input:

```
import numpy as np
arr_a = np.array([10, 20, 30])
arr_b = np.array([1, 2, 3])
print("Element-wise Subtraction:", arr_a - arr_b)
```

Output:

```
Element-wise Subtraction: [ 9 18 27]
```

Question 3: Write a program to multiply two arrays element-wise.

Input:

```
import numpy as np
arr_a = np.array([2, 3, 4])
arr_b = np.array([5, 6, 7])
print("Element-wise Multiplication:", arr_a * arr_b)
```

Output:

```
Element-wise Multiplication: [10 18 28]
```

Question 4: Write a program to divide two arrays element-wise.

Input:

```
import numpy as np
arr_a = np.array([10, 20, 30])
arr_b = np.array([2, 4, 5])
print("Element-wise Division:", arr_a / arr_b)
```

Output:

```
Element-wise Division: [5. 5. 6.]
```

7. Universal Functions (ufuncs)

7.1 np.sqrt()

`np.sqrt()` calculates the square root of every element in an array at once. It is one of many fast, vectorized math functions called ufuncs.

Question 1: Write a program to calculate the square root of every element in an array.

Input:

```
import numpy as np
numbers = np.array([0, 1, 4])
print("Calculated Square Roots:", np.sqrt(numbers))
```

Output:

```
Calculated Square Roots: [0. 1. 2.]
```

Question 2: Write a program to calculate the square roots of a list of perfect squares.

Input:

```
import numpy as np
numbers = np.array([9, 16, 25])
print(np.sqrt(numbers))
```

Output:

```
[3.  4.  5.]
```

Question 3: Write a program to calculate the square roots of non-perfect-square numbers.

Input:

```
import numpy as np
numbers = np.array([2, 3])
print(np.sqrt(numbers))
```

Output:

```
[1.41421356 1.73205081]
```

Question 4: Write a program to calculate the square root of every element in a 2D array.

Input:

```
import numpy as np
matrix = np.array([[4, 9], [16, 25]])
print(np.sqrt(matrix))
```

Output:

```
[[2.  3.]
 [4.  5.]]
```

7.2 np.exp() & np.log()

np.exp() raises the constant e to the power of each element, while np.log() computes the natural logarithm of each element. Both run across the whole array in one call.

Question 1: Write a program to calculate the exponential and natural log of an array.

Input:

```
import numpy as np
arr = np.array([1, 2])
print("Exponential:", np.exp(arr))
print("Natural Log:", np.log(arr))
```

Output:

```
Exponential: [2.71828183 7.3890561 ]
Natural Log: [0. 0.69314718]
```

Question 2: Write a program to calculate the exponential of an array starting from zero.

Input:

```
import numpy as np
arr = np.array([0, 1, 2])
print(np.exp(arr))
```

Output:

```
[1. 2.71828183 7.3890561 ]
```

Question 3: Write a program to calculate the natural log of powers of e.

Input:

```
import numpy as np
arr = np.array([1, np.e, np.e**2])
print(np.log(arr))
```

Output:

```
[0. 1. 2.]
```

Question 4: Write a program to calculate the base-10 logarithm of an array.

Input:

```
import numpy as np
arr = np.array([10, 100])
print(np.log10(arr))
```

Output:

```
[1. 2.]
```

8. Statistical Functions

8.1 np.sum() & np.mean()

np.sum() adds up all the values in an array, and np.mean() divides that sum by the number of elements to get the average. Both can also be applied along a single axis of a matrix.

Question 1: Write a program to calculate the sum and mean of an array.

Input:

```
import numpy as np
data = np.array([10, 50, 20, 40])
print("Sum:", np.sum(data))
print("Mean:", np.mean(data))
```

Output:

```
Sum: 120
Mean: 30.0
```

Question 2: Write a program to calculate the sum of a simple array.

Input:

```
import numpy as np
data = np.array([1, 2, 3, 4, 5])
print("Sum:", np.sum(data))
```

Output:

```
Sum: 15
```

Question 3: Write a program to calculate the column-wise sum of a matrix.

Input:

```
import numpy as np
matrix = np.array([[1, 2], [3, 4]])
print("Column Sums:", np.sum(matrix, axis=0))
```

Output:

Column Sums: [4 6]

Question 4: Write a program to calculate the row-wise mean of a matrix.

Input:

```
import numpy as np
matrix = np.array([[1, 2], [3, 4]])
print("Row Means:", np.mean(matrix, axis=1))
```

Output:

Row Means: [1.5 3.5]

8.2 np.min(), np.max(), np.argmin(), np.argmax()

np.min() and np.max() report the smallest and largest values in an array, while np.argmin() and np.argmax() report the index positions where those values occur.

Question 1: Write a program to find the minimum value and the index of the maximum value in an array.

Input:

```
import numpy as np
scores = np.array([45, 89, 23, 71])
print("Minimum Value:", np.min(scores))
print("Index of Max Value:", np.argmax(scores))
```

Output:

Minimum Value: 23
Index of Max Value: 1

Question 2: Write a program to find the maximum value in an array.

Input:

```
import numpy as np
scores = np.array([45, 89, 23, 71])
print("Maximum Value:", np.max(scores))
```

Output:

Maximum Value: 89

Question 3: Write a program to find the index of the minimum value in an array.

Input:

```
import numpy as np
scores = np.array([45, 89, 23, 71])
print("Index of Min Value:", np.argmin(scores))
```

Output:

```
Index of Min Value: 2
```

Question 4: Write a program to find the maximum value of each column in a matrix.

Input:

```
import numpy as np
matrix = np.array([[3, 7], [9, 1]])
print("Max of Each Column:", np.max(matrix, axis=0))
```

Output:

```
Max of Each Column: [9 7]
```

9. Manipulation Functions

9.1 np.reshape()

reshape() rearranges an array's data into a new shape without changing the underlying values or their order. The total number of elements must stay the same before and after.

Question 1: Write a program to reshape a 1D array into a 3x2 matrix.

Input:

```
import numpy as np
flat_arr = np.array([1, 2, 3, 4, 5, 6])
reshaped_matrix = flat_arr.reshape(3, 2)
print("Reshaped Matrix:\n", reshaped_matrix)
```

Output:

```
Reshaped Matrix:
[[1 2]
```

```
[3 4]
[5 6]]
```

Question 2: Write a program to reshape a generated range of numbers into a 2x3 matrix.

Input:

```
import numpy as np
flat_arr = np.arange(6)
print(flat_arr.reshape(2, 3))
```

Output:

```
[[0 1 2]
 [3 4 5]]
```

Question 3: Write a program to reshape a generated range of numbers into a 3x4 matrix.

Input:

```
import numpy as np
flat_arr = np.arange(12)
print(flat_arr.reshape(3, 4))
```

Output:

```
[[ 0 1 2 3]
 [ 4 5 6 7]
 [ 8 9 10 11]]
```

Question 4: Write a program to reshape an array using an inferred dimension.

Input:

```
import numpy as np
flat_arr = np.arange(8)
print(flat_arr.reshape(2, -1))
```

Output:

```
[[0 1 2 3]
 [4 5 6 7]]
```

9.2 flatten() & ravel()

flatten() and ravel() both collapse a multi-dimensional array into a single 1D array. flatten() always returns a copy, while ravel() returns a view whenever possible.

Question 1: Write a program to flatten a 2D array into a 1D array using flatten().

Input:

```
import numpy as np
matrix = np.array([[1, 2], [3, 4]])
flat = matrix.flatten()
print("Flattened 1D Array:", flat)
```

Output:

```
Flattened 1D Array: [1 2 3 4]
```

Question 2: Write a program to flatten a 2D array into a 1D array using ravel().

Input:

```
import numpy as np
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix.ravel())
```

Output:

```
[1 2 3 4 5 6]
```

Question 3: Write a program to flatten a taller 2D array using flatten().

Input:

```
import numpy as np
matrix = np.array([[1, 2], [3, 4], [5, 6]])
print(matrix.flatten())
```

Output:

```
[1 2 3 4 5 6]
```

Question 4: Write a program to flatten a 3D array using ravel().

Input:

```
import numpy as np
cube = np.array([[[1, 2]], [[3, 4]]])
print(cube.ravel())
```

Output:

```
[1 2 3 4]
```

9.3 T (Transpose)

The .T attribute flips a matrix so that its rows become columns and its columns become rows. It is a quick way to swap the axes of a 2D array.

Question 1: Write a program to print the shape of a transposed matrix.

Input:

```
import numpy as np
matrix = np.array([[1, 2, 3], [4, 5, 6]])
transposed = matrix.T
print("Transposed Shape:", transposed.shape)
```

Output:

```
Transposed Shape: (3, 2)
```

Question 2: Write a program to transpose a square matrix.

Input:

```
import numpy as np
matrix = np.array([[1, 2], [3, 4]])
print(matrix.T)
```

Output:

```
[[1 3]
 [2 4]]
```

Question 3: Write a program to transpose a single-row matrix into a column.

Input:

```
import numpy as np
row = np.array([[1, 2, 3]])
```

```
print(row.T)
```

Output:

```
[[1]  
 [2]  
 [3]]
```

Question 4: Write a program to transpose a rectangular matrix.

Input:

```
import numpy as np  
matrix = np.array([[1, 2, 3], [4, 5, 6]])  
print(matrix.T)
```

Output:

```
[[1 4]  
 [2 5]  
 [3 6]]
```

10. Stacking Arrays

10.1 np.vstack() & np.hstack()

`np.vstack()` stacks arrays on top of each other to add more rows, and `np.hstack()` joins arrays side by side to add more columns. Both require the arrays to have compatible shapes.

Question 1: Write a program to stack two arrays vertically and horizontally.

Input:

```
import numpy as np  
row1 = np.array([1, 2])  
row2 = np.array([3, 4])  
print("Vertical Stack:\n", np.vstack((row1, row2)))  
print("Horizontal Stack:", np.hstack((row1, row2)))
```

Output:

Vertical Stack:

```
[[1 2]
 [3 4]]
```

Horizontal Stack: [1 2 3 4]

Question 2: Write a program to stack two 1D arrays vertically into a matrix.

Input:

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.vstack((a, b)))
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

Question 3: Write a program to stack two column arrays horizontally.

Input:

```
import numpy as np
a = np.array([[1], [2]])
b = np.array([[3], [4]])
print(np.hstack((a, b)))
```

Output:

```
[[1 3]
 [2 4]]
```

Question 4: Write a program to stack three 1D arrays vertically.

Input:

```
import numpy as np
a = np.array([1, 2])
b = np.array([3, 4])
c = np.array([5, 6])
print(np.vstack((a, b, c)))
```

Output:

```
[[1 2]
 [3 4]
 [5 6]]
```

11. Splitting Arrays

11.1 np.split()

`np.split()` cuts a single array into a chosen number of equally sized smaller arrays. The array's length must divide evenly by the number of requested parts.

Question 1: Write a program to split an array into two equal parts and print the first part.

Input:

```
import numpy as np
dataset = np.array([10, 20, 30, 40, 50, 60])
part1, part2 = np.split(dataset, 2)
print("Part 1:", part1)
```

Output:

```
Part 1: [10 20 30]
```

Question 2: Write a program to split an array into two equal parts and print the second part.

Input:

```
import numpy as np
dataset = np.array([10, 20, 30, 40, 50, 60])
part1, part2 = np.split(dataset, 2)
print("Part 2:", part2)
```

Output:

```
Part 2: [40 50 60]
```

Question 3: Write a program to split a generated range of numbers into three equal parts.

Input:

```
import numpy as np
dataset = np.arange(9)
parts = np.split(dataset, 3)
print(parts)
```

Output:

```
[array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]
```

Question 4: Write a program to split a matrix into a top half and a bottom half.

Input:

```
import numpy as np
matrix = np.arange(16).reshape(4, 4)
top, bottom = np.split(matrix, 2)
print("Top Half:\n", top)
```

Output:

```
Top Half:
[[0 1 2 3]
 [4 5 6 7]]
```

12. Broadcasting

12.1 Scalar Broadcasting Example

Broadcasting lets NumPy automatically stretch a smaller array so it can combine with a larger array in an operation. A single scalar value is broadcast across every element of the target array.

Question 1: Write a program to multiply every element of an array by a scalar using broadcasting.

Input:

```
import numpy as np
vector = np.array([2, 4, 6])
broadcasted_out = vector * 10
print("Broadcasting Scaling Product:", broadcasted_out)
```

Output:

Broadcasting Scaling Product: [20 40 60]

Question 2: Write a program to add a scalar to every element of an array using broadcasting.

Input:

```
import numpy as np
vector = np.array([1, 2, 3])
print(vector + 100)
```

Output:

[101 102 103]

Question 3: Write a program to add a scalar to every element of a 2D array using broadcasting.

Input:

```
import numpy as np
matrix = np.array([[1, 2], [3, 4]])
print(matrix + 1)
```

Output:

[[2 3]
[4 5]]

Question 4: Write a program to add a 1D row vector to every row of a 2D matrix using broadcasting.

Input:

```
import numpy as np
matrix = np.array([[1, 2, 3], [4, 5, 6]])
row_vector = np.array([10, 20, 30])
print(matrix + row_vector)
```

Output:

[[11 22 33]
[14 25 36]]

13. Copy vs. View

13.1 copy() vs view()

`copy()` creates a fully independent array with its own data, so changes to it never affect the original. `view()` creates a new array object that shares the same underlying data, so changes to one appear in the other.

Question 1: Write a program to show that modifying a copied array does not affect the original.

Input:

```
import numpy as np
original = np.array([1, 2, 3])
cloned_copy = original.copy()
view_mirror = original.view()
cloned_copy[0] = 999
print("Original Array after Copy Mod:", original)
```

Output:

```
Original Array after Copy Mod: [1 2 3]
```

Question 2: Write a program to show that modifying a view affects the original array.

Input:

```
import numpy as np
original = np.array([1, 2, 3])
view_mirror = original.view()
view_mirror[0] = 999
print("Original Array after View Mod:", original)
```

Output:

```
Original Array after View Mod: [999 2 3]
```

Question 3: Write a program to check whether a copied array shares memory with the original.

Input:

```
import numpy as np
original = np.array([5, 5, 5])
cloned_copy = original.copy()
print("Copy shares memory with original:",
      np.shares_memory(original, cloned_copy))
```

Output:

```
Copy shares memory with original: False
```

Question 4: Write a program to check whether a view shares memory with the original array.

Input:

```
import numpy as np
original = np.array([5, 5, 5])
view_mirror = original.view()
print("View shares memory with original:",
      np.shares_memory(original, view_mirror))
```

Output:

```
View shares memory with original: True
```

14. Random Numbers

14.1 np.random.randint() & np.random.rand()

`np.random.randint()` generates random whole numbers within a given range, while `np.random.rand()` generates random decimal numbers between 0 and 1. A fixed seed can be set to make results repeatable.

Question 1: Write a program to generate random integers and random floats with a fixed seed.

Input:

```
import numpy as np
np.random.seed(0)
random_integers = np.random.randint(1, 10, size=3)
random_floats = np.random.rand(2)
print("Random Integers:", random_integers)
```

```
print("Random Floats:", random_floats)
```

Output:

```
Random Integers: [6 1 4]  
Random Floats: [0.85794562 0.84725174]
```

Question 2: Write a program to generate five random integers between 0 and 100 with a fixed seed.

Input:

```
import numpy as np  
np.random.seed(1)  
print(np.random.randint(0, 100, size=5))
```

Output:

```
[37 12 72 9 75]
```

Question 3: Write a program to generate a 3x2 matrix of random floats with a fixed seed.

Input:

```
import numpy as np  
np.random.seed(2)  
print(np.random.rand(3, 2))
```

Output:

```
[[0.4359949 0.02592623]  
 [0.54966248 0.43532239]  
 [0.4203678 0.33033482]]
```

Question 4: Write a program to generate a 2x2 matrix of small random integers with a fixed seed.

Input:

```
import numpy as np  
np.random.seed(3)  
print(np.random.randint(5, size=(2, 2)))
```

Output:

```
[[2 0]
 [1 3]]
```

15. Searching & Sorting

15.1 np.where()

`np.where()` scans an array against a condition and returns the index positions where that condition is True. It can also be used to pick values from two arrays based on the condition.

Question 1: Write a program to find the indices of array elements greater than 10.

Input:

```
import numpy as np
arr_data = np.array([8, 12, 3, 25])
found_indexes = np.where(arr_data > 10)
print("Indices Matching Criteria:", found_indexes[0])
```

Output:

```
Indices Matching Criteria: [1 3]
```

Question 2: Write a program to find the indices of the even numbers in an array.

Input:

```
import numpy as np
arr_data = np.array([1, 2, 3, 4, 5])
print(np.where(arr_data % 2 == 0)[0])
```

Output:

```
[1 3]
```

Question 3: Write a program to label array elements as big or small based on a condition.

Input:

```
import numpy as np
arr_data = np.array([1, 2, 3, 4])
print(np.where(arr_data > 2, "big", "small"))
```

Output:

```
['small' 'small' 'big' 'big']
```

Question 4: Write a program to replace negative values in an array with zero.

Input:

```
import numpy as np
arr_data = np.array([-1, 5, -3, 8])
print(np.where(arr_data < 0, 0, arr_data))
```

Output:

```
[0 5 0 8]
```

15.2 np.sort()

np.sort() returns a new array with all the values arranged in ascending order by default. The original array is left unmodified.

Question 1: Write a program to sort an array of numbers in ascending order.

Input:

```
import numpy as np
unsorted = np.array([45, 12, 89, 3])
sorted_arr = np.sort(unsorted)
print("Sorted Array Output:", sorted_arr)
```

Output:

```
Sorted Array Output: [ 3 12 45 89]
```

Question 2: Write a program to sort a small array of integers.

Input:

```
import numpy as np
unsorted = np.array([5, 1, 4, 2, 3])
print(np.sort(unsorted))
```

Output:

```
[1 2 3 4 5]
```

Question 3: Write a program to sort an array and then reverse it into descending order.

Input:

```
import numpy as np
unsorted = np.array([9, 7, 5, 3])
print(np.sort(unsorted)[::-1])
```

Output:

```
[9 7 5 3]
```

Question 4: Write a program to sort a matrix column-wise.

Input:

```
import numpy as np
matrix = np.array([[3, 1], [2, 4]])
print(np.sort(matrix, axis=0))
```

Output:

```
[[2 1]
 [3 4]]
```

16. Matrix Multiplication

16.1 np.dot() & @ Operator

np.dot() and the @ operator both perform standard matrix multiplication between two arrays. They follow the linear algebra rule that inner dimensions must match.

Question 1: Write a program to multiply two matrices using np.dot().

Input:

```
import numpy as np
matrix_a = np.array([[1, 2]])
```

```
matrix_b = np.array([[3], [4]])  
print("Product Result via dot():\n", np.dot(matrix_a, matrix_b))
```

Output:

```
Product Result via dot():  
[[11]]
```

Question 2: Write a program to multiply two matrices using the @ operator.

Input:

```
import numpy as np  
matrix_a = np.array([[1, 2]])  
matrix_b = np.array([[3], [4]])  
print("Product Result via @:\n", matrix_a @ matrix_b)
```

Output:

```
Product Result via @:  
[[11]]
```

Question 3: Write a program to multiply two 2x2 matrices using np.dot().

Input:

```
import numpy as np  
a = np.array([[1, 2], [3, 4]])  
b = np.array([[5, 6], [7, 8]])  
print(np.dot(a, b))
```

Output:

```
[[19 22]  
 [43 50]]
```

Question 4: Write a program to compute the dot product of two vectors.

Input:

```
import numpy as np  
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])  
print("Dot Product of Vectors:", np.dot(a, b))
```

Output:

Dot Product of Vectors: 32

17. File Handling

17.1 np.savetxt() & np.loadtxt()

`np.savetxt()` writes an array out to a plain text or CSV file on disk, and `np.loadtxt()` reads that file back into memory as an array. This is a simple way to persist array data between program runs.

Question 1: Write a program to save a 2D array to a CSV file and print its shape after reloading.

Input:

```
import numpy as np
export_data = np.array([[5, 10], [15, 20]])
np.savetxt('data_log.csv', export_data, delimiter=',')
imported_data = np.loadtxt('data_log.csv', delimiter=',')
print("Imported Shape Vector:", imported_data.shape)
```

Output:

Imported Shape Vector: (2, 2)

Question 2: Write a program to save a 1D array to a CSV file and load it back.

Input:

```
import numpy as np
export_data = np.array([1, 2, 3, 4])
np.savetxt('vector_log.csv', export_data, delimiter=',')
print("Loaded Back:", np.loadtxt('vector_log.csv', delimiter=','))
```

Output:

Loaded Back: [1. 2. 3. 4.]

Question 3: Write a program to save a float array to CSV with two decimal places and load it back.

Input:

```
import numpy as np
export_data = np.array([[1.5, 2.5], [3.5, 4.5]])
np.savetxt('floats_log.csv', export_data, delimiter=',', fmt='%.2f')
print("Loaded Floats:\n", np.loadtxt('floats_log.csv',
delimiter=','))
```

Output:

```
Loaded Floats:
[[1.5 2.5]
 [3.5 4.5]]
```

Question 4: Write a program to save a reshaped range of numbers to CSV and load it back.

Input:

```
import numpy as np
export_data = np.arange(6).reshape(2, 3)
np.savetxt('grid_log.csv', export_data, delimiter=',')
print("Loaded Grid:\n", np.loadtxt('grid_log.csv', delimiter=','))
```

Output:

```
Loaded Grid:
[[0. 1. 2.]
 [3. 4. 5.]]
```